NewMessage, 128, 130, 172–174
testing, 172–174
updating, 109
Channels, 119, 139
classes vs. structs, 43
comparison operators, 16
compilation environment macro, 42
compile time vs. runtime, 30–31
compiled vs. interpreted languages, 8
components, Phoenix, 118, 134–138
comprehensions, 45–47
concurrency, *see also* GenServer library
in Elixir, 69
HTTP clients, 144
vs. parallelism, 66–70
in Ruby, 65, 67
cond statements, 59
CondClauseError, 60
connection pooling, HTTP requests, 142
contact data, persisting, 105–107
context modules for functions, 112–114
control flow, 49, 55–60
controller actions, 125–131
Cowboy web server, 119
create action, 129
create index(:table_name, [:field_one, :field_two]) function, 99
create table(:name) do / end function, 98
CRON plugin, 168

D

data access libraries, “magic” element, 91
data layer, *see* Ecto library
data mutability, 14, 39
data persistence, *see* Ecto library
data pipelines, writing with job systems, 155
data structures, 33–47
enumerating, 43–47
implementing, 41–43
lists, 16, 33–37
maps, 16, 37–41
pattern matching, 52–54
data types, 14–16
database, querying, *see* queries
def macro, 20
default-value keys, structs, 40
defdelegate, 113
defmodule macro, 20
Dialyzer package, 191
digesting pipeline assets, 134
do block, 94
domain-specific language (DSL), 93
double backslash (\\) operator, 24
double hyphen (--), lists, 34
double pluses (++)
list concatenation, 34
list pattern matching, 52–53

E

Ecto library, 87–115
changesets to persist data, 101–104
context modules for functions, 112–114
foundations of, 91–93
and GenServers, 66
migrations to create database tables, 98–101
and Phoenix, 119
query data with Ecto.Query, 105–112
sample project to build, 88–90
schema construction, 93–97
testing queries, 175–176
Ecto.Changeset, 101, 109, 136
Ecto.Migration module, 98
Ecto.Query, 105–112
Ecto.Repomodule, 105
Ecto.Schema module, 93–95
ecto_sql library, 98
[element | list], 34
Elixir, 5–12, 14, *see also* syntax for Elixir
BEAM, 9
case for, 5–7
concurrency and parallelism models, 69–70
deployment, 188
Erlang/OTP, 8, 10
function orientation of, 14, 41
installing, 10–11
writing first code process, 11–12
else clauses, with statement, 58
else if syntax, Elixir’s lack of, 56
email sending use case, async jobs, 154
Endpoint, 122
endpoints, testing, 179
enqueued jobs, testing for, 183
Enum module, 43
Enum.each/2, 44
Enum.map/2, 44
Enum.reduce/3, 45
Enumerable protocol, 43
Erlang/OTP, 8, 10, 143, 191
error isolation, 73
esbuild asset bundler, 132
exact-position basis for pattern matching, 52
exception operators, 17
exclamation mark (!), function error, 35
external API requests, testing, 176–179
ExUnit, 172–174

F

field macro function, 94
fields, schema, 95
Finch HTTP request library, 144
Flash messages, 130
float data type, 15
fn keyword, 25
for keyword, 45
function-oriented language, Elixir as, 14, 41
FunctionClauseError, 62
functions, 16–27
anonymous, 12, 18, 22, 25
capturing, 25–27